

AI

AI Deployment as a Service

End-to-end framework — agents, accelerators and implementation plan

v1.0 Framework

01
Pre-Deploy

02
Deploy

03
Validate

04
Operate

05
Support

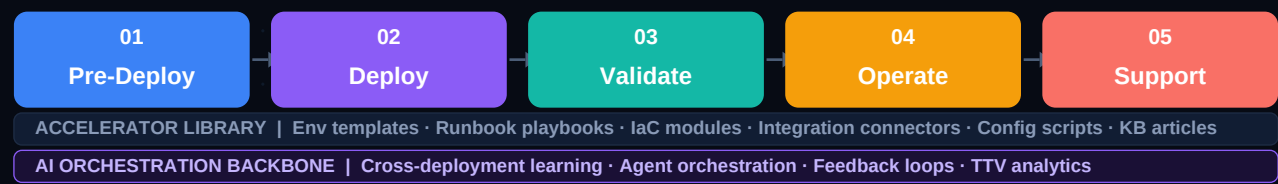
15
AI agents built

5
Deployment phases

16 weeks
To full deployment

∞
Deployments scalable

TABLE OF CONTENTS



- 01 Pre-Deploy Phase (3 Agents)
- 02 Deploy Phase (3 Agents)
- 03 Validate Phase (3 Agents)
- 04 Operate Phase (3 Agents)
- 05 Support Phase (3 Agents)
- 06 Implementation Roadmap (16 Weeks)

Intelligence gathering before a single script runs. Most TTV is won or lost here — catching conflicts in pre-deploy takes minutes; catching them during deployment takes days.

ES

Environment Scanner Agent

Scans customer infrastructure and produces a structured compatibility report against product requ...

INPUTS	OUTPUTS
■ Customer env credentials / access config	■ Environment fingerprint JSON
■ Product requirements spec (product_req.json)	■ Compatibility report (pass/fail per requirement)
■ Network topology / cloud account info	■ Conflict list with remediation actions
	■ Recommended environment adjustments

CODE SNIPPET · Claude Code (claude-sonnet-4-20250514)

```
// env_scanner_agent.js – reusable across all customers
import Anthropic from '@anthropic-ai/sdk';

export async function runEnvScanner({ envType, productReqPath, credsRef }) {
  const client = new Anthropic();
  const req = await fs.readFile(productReqPath, 'utf8');

  const res = await client.messages.create({
    model: 'claude-sonnet-4-20250514',
    max_tokens: 4096,
    system: SYSTEM_PROMPT, // returns structured JSON only
    tools: [BASH_TOOL, FILE_READ_TOOL],
    messages: [{ role: 'user', content:
      `Env type: ${envType}\nRequirements: ${req}\nScan and report.`
    }]
  });

  return parseJSON(res);
  // Output: { compatible, env_fingerprint, conflicts[], recommendations[] }
}
```

REUSABILITY DESIGN

- Fully reusable. Product requirements live in a separate JSON file — add new products without touching agent code. The `env_type` parameter selects the correct cloud adapter (AWS / Azure / GCP / on-prem). Output schema is fixed so all downstream agents always receive the same structure.

DEPENDENCIES

@anthropic-ai/sdk	Cloud CLI (aws / az / gcloud)	product_req.json	Vault / secrets manager
-------------------	-------------------------------	------------------	-------------------------

DM

Dependency Mapper Agent

Traces all software, service and network dependencies — including transitive ones — producing a q...

INPUTS	OUTPUTS
■ Application manifests (package.json, requirements.txt, pom.xml)	■ Dependency graph JSON
■ Installed software inventory (from Scanner)	■ Version conflict matrix
■ Product dependency spec	■ Missing dependencies list
	■ Upgrade path recommendations

CODE SNIPPET · Claude Code (claude-sonnet-4-20250514)

```
// dep_mapper_agent.js
export async function runDepMapper({ manifestPaths, softwareInventory, productId }) {
  const client = new Anthropic();
  const manifests = await loadManifests(manifestPaths);
  const depSpec = await loadDepSpec(productId); // per-product config

  const res = await client.messages.create({
    model: 'claude-sonnet-4-20250514', max_tokens: 4096,
    system: 'Parse manifests, build dependency graph, identify version' +
      ' conflicts and missing deps. Return ONLY valid JSON.',
    messages: [{ role: 'user', content:
      `Manifests: ${JSON.stringify(manifests)}\nInventory: ${JSON.stringify(softwareInventory)}\n
      Dep spec: ${JSON.stringify(depSpec)}`
    }
  ]});

  return JSON.parse(res);
}
```

REUSABILITY DESIGN

- Product dependency specs are separate config files. The mapper accepts any manifest format — Claude parses them, so adding support for a new format requires no code change, only testing.

DEPENDENCIES

@anthropic-ai/sdk	product_dep_spec.json	Scanner output (env_fingerprint)
-------------------	-----------------------	----------------------------------

IS

Infra Sizer Agent

Recommends compute, storage and network sizing across three tiers based on expected load and hist...

INPUTS	OUTPUTS
■ Expected user count and data volume	■ Sizing tiers JSON (min / recommended / production)
■ Usage pattern profile	■ Cost estimate per tier
■ product_benchmarks.json	■ Auto-scaling thresholds and triggers
■ Historical deployment data store	

CODE SNIPPET · Claude Code (claude-sonnet-4-20250514)

```
// infra_sizer_agent.js
export async function runInfraSizer({ userCount, dataGb, usageProfile, productId }) {
  const client = new Anthropic();
  const benchmarks = await loadBenchmarks(productId);
  const historical = await queryHistorical({ productId });

  const res = await client.messages.create({
    model: 'claude-sonnet-4-20250514', max_tokens: 2048,
    system: 'Calculate resource requirements. Return JSON with three tiers.',
    messages: [{ role: 'user', content:
      `Users: ${userCount} Data: ${dataGb}GB\n` +
      `Profile: ${JSON.stringify(usageProfile)}\n` +
      `Benchmarks: ${JSON.stringify(benchmarks)}`
    }]
  });

  return parseJSON(res);
}
```

REUSABILITY DESIGN

- Product benchmarks are per-product JSON configs. Historical data is aggregated centrally from all deployments — every deployment enriches it, making future recommendations more accurate.

DEPENDENCIES

@anthropic-ai/sdk	product_benchmarks.js n	historical_deployments _db	cloud_pricing_api
-------------------	----------------------------	-------------------------------	-------------------

The deployment engine. AI handles configuration translation and IaC composition — humans approve at critical gates only. Target: zero manual config edits per deployment.

CA

Config Adapter Agent

Adapts product configuration templates to the target environment, handling cloud translation, net...

INPUTS	OUTPUTS
■ Base config templates (YAML / JSON)	■ Environment-specific config files
■ Environment fingerprint from Scanner	■ Config diff report (what changed and why)
■ Customer override spec (optional)	■ Validation checksum for audit trail

CODE SNIPPET · Claude Code (claude-sonnet-4-20250514)

```
// config_adapter_agent.js
export async function runConfigAdapter({ templatePath, envFingerprint, overrides = {} }) {
  const client = new Anthropic();
  const baseConfig = await fs.readFile(templatePath, 'utf8');

  const res = await client.messages.create({
    model: 'claude-sonnet-4-20250514', max_tokens: 4096,
    system: 'Adapt the base config for the target environment.' +
      ' Translate cloud values, network ranges, endpoint URLs.' +
      ' Return adapted YAML then a JSON diff explaining each change.',
    messages: [{ role: 'user', content:
      `Base config:\n${baseConfig}\n\n` +
      `Target env: ${JSON.stringify(envFingerprint)}\n` +
      `Overrides: ${JSON.stringify(overrides)}`
    }]
  });

  const { adaptedConfig, diff } = parseConfigResponse(res);
  await fs.writeFile(outputPath, adaptedConfig);
  return { path: outputPath, diff, checksum: hash(adaptedConfig) };
}
```

REUSABILITY DESIGN

- Config templates are per-product. The adapter logic is fully reusable — it reads the fixed `env_fingerprint` schema and the per-product template. Adding a new product = new template directory, no agent code change.

DEPENDENCIES

@anthropic-ai/sdk	Config templates library	Scanner output	Customer override spec
-------------------	--------------------------	----------------	------------------------

IG

IaC Generator Agent

Selects and parameterises IaC modules from the accelerator library based on environment type and ...

INPUTS

- Infra sizing recommendation (recommended tier)
- Environment type and cloud provider
- Product IaC module catalog

OUTPUTS

- Ready-to-apply Terraform / Pulumi / Ansible files
- Module selection rationale
- Apply sequence plan (dependency-ordered)

CODE SNIPPET · Claude Code (claude-sonnet-4-20250514)

```
// iac_generator_agent.js
export async function runIaCGenerator({ sizingRec, envType, cloudProvider, productId }) {
  const client = new Anthropic();
  const catalog = await loadModuleCatalog(productId, cloudProvider);

  const res = await client.messages.create({
    model: 'claude-sonnet-4-20250514', max_tokens: 8192,
    system: 'Select modules from the catalog and parameterise them.' +
      ' Output valid Terraform HCL + JSON apply_sequence.',
    tools: [FILE_CREATE_TOOL],
    messages: [{ role: 'user', content:
      `Sizing: ${JSON.stringify(sizingRec.recommended)}\n` +
      `Env: ${envType} Cloud: ${cloudProvider}\n` +
      `Catalog: ${JSON.stringify(catalog)}`
    }]
  });

  return extractIaCOutput(res);
}
```

REUSABILITY DESIGN

- Module catalogs are versioned per-product × per-cloud-provider. Adding support for a new cloud = new catalog directory. The generator selects the right catalog via productId + cloudProvider params.

DEPENDENCIES

@anthropic-ai/sdk

IaC module catalog

Infra Sizer output

Terraform / Ansible
CLI

PO

Provisioning Orchestrator

Orchestrates the end-to-end provisioning sequence with rollback intelligence and human approval g...

INPUTS

- IaC files from Generator

OUTPUTS

- Provisioning logs (structured JSON)

■ Deployment plan YAML	■ Resource inventory
■ Approval gate config (who approves, via what channel)	■ Deployment receipt
	■ Rollback state if triggered

CODE SNIPPET · Claude Code (claude-sonnet-4-20250514)

```
// provisioning_orchestrator.js
export async function runProvisioning({ iacPath, deployPlan, approvalConfig }) {
  const client = new Anthropic();
  const state = { steps: [], resources: [], errors: [] };

  for (const step of deployPlan.steps) {
    if (step.requires_approval) await requestApproval(step, approvalConfig);
    try {
      const result = await executeStep(step, iacPath);
      state.steps.push({ id: step.id, result, ts: Date.now() });
    } catch (err) {
      // AI decides: retry | skip | rollback
      const decision = await client.messages.create({
        model: 'claude-sonnet-4-20250514', max_tokens: 512,
        system: 'Evaluate error. Return JSON: { action, reason }.',
        messages: [{ role: 'user', content:
          `Step: ${step.id} Error: ${err.message}`
        }]
      });
      await handleDecision(parseJSON(decision), state, step);
    }
  }
  return state;
}
```

REUSABILITY DESIGN

■ Deployment plan schema is standardised. Product-specific steps are declared in plan YAML files, not in the orchestrator code. Add new products by adding new plan templates.

DEPENDENCIES

@anthropic-ai/sdk	IaC Generator output	deployment_plan.yaml	Approval service (Slack / email)
-------------------	----------------------	----------------------	----------------------------------

Automated proof that the deployment is production-ready. Humans review exceptions, not everything — the AI handles all baseline checking and generates the evidence package.

ST

Smoke Test Runner Agent

Generates and executes smoke tests post-deployment, covering all critical user journeys derived f...

INPUTS	OUTPUTS
■ Product API spec (OpenAPI / Swagger)	■ Test results JSON (pass/fail per scenario)
■ Deployment config (endpoint URLs, auth tokens)	■ Anomaly list with context
■ Test scenario templates per product	■ Performance baseline snapshot

CODE SNIPPET · Claude Code (claude-sonnet-4-20250514)

```
// smoke_test_runner.js
export async function runSmokeTests({ apiSpecPath, deployConfig, productId }) {
  const client = new Anthropic();
  const apiSpec = await fs.readFile(apiSpecPath, 'utf8');
  const templates = await loadScenarioTemplates(productId);

  // Step 1: Generate test cases from API spec
  const tcRes = await client.messages.create({
    model: 'claude-sonnet-4-20250514', max_tokens: 4096,
    system: 'Generate smoke test cases. Return JSON array:' +
      ' [{ id, name, endpoint, method, body, expected_status }]',
    messages: [{ role: "user", content: `${apiSpec}\nTemplates: ${JSON.stringify(templates)}` } ]
  });

  // Step 2: Execute and analyse
  const cases = parseJSON(tcRes);
  const results = await Promise.all(cases.map(tc => executeTest(tc, deployConfig)));
  const summary = await analyseResults(client, results);
  return { results, summary };
}
```

REUSABILITY DESIGN

- API spec and scenario templates are per-product. The execution and analysis engine is shared. Adding a new product means providing its OpenAPI spec and any additional scenario templates — no agent code changes.

DEPENDENCIES

@anthropic-ai/sdk	Product OpenAPI spec	Scenario templates	HTTP client
-------------------	----------------------	--------------------	-------------

RE

Runbook Executor Agent

Executes validation runbooks step-by-step with structured logging, AI-evaluated outcomes and huma...

INPUTS	OUTPUTS
■ Runbook YAML definition	■ Execution log (step-by-step JSON)
■ Deployment context (env, config)	■ Step outcomes with evidence
■ Approval configuration	■ Runbook completion certificate

CODE SNIPPET · Claude Code (claude-sonnet-4-20250514)

```
// runbook_executor.js
export async function executeRunbook({ runbookPath, deploymentCtx, approvalConfig }) {
  const client = new Anthropic();
  const runbook = yaml.load(await fs.readFile(runbookPath, 'utf8'));
  const log = { id: runbook.id, steps: [], started_at: Date.now() };

  for (const step of runbook.steps) {
    if (step.type === 'approval') {
      log.steps.push(await requestApproval(step, approvalConfig));
    } else {
      const rawResult = await executeCheck(step, deploymentCtx);
      const evaluation = await client.messages.create({
        model: 'claude-sonnet-4-20250514', max_tokens: 256,
        system: 'Evaluate check. Return JSON: { passed, evidence, notes }',
        messages: [{ role: "user", content:
          `Criteria: ${step.criteria}\nResult: ${JSON.stringify(rawResult)}`
        }]
      });
      const outcome = parseJSON(evaluation);
      log.steps.push({ ...outcome, step_id: step.id });
      if (!outcome.passed && step.blocking) break;
    }
  }
  return log;
}
```

REUSABILITY DESIGN

- Runbook YAML format is standardised. Runbooks are per-product and per-scenario (install, upgrade, migration, DR). The executor handles any conformant runbook.

DEPENDENCIES

@anthropic-ai/sdk	Runbook YAML library	Deployment context	Approval notification service
-------------------	----------------------	--------------------	-------------------------------



Sign-off Gate Validator

Checks all go-live criteria and generates a digital evidence package for customer sign-off — maki...

INPUTS	OUTPUTS
■ Smoke test results	■ Go/no-go decision with full rationale
■ Runbook execution logs	■ Evidence package (PDF + JSON)
■ Customer acceptance criteria JSON	■ Outstanding items list
■ Performance benchmark targets	■ Customer sign-off document

CODE SNIPPET · Claude Code (claude-sonnet-4-20250514)

```
// signoff_gate_validator.js
export async function runSignoff({ testResults, runbookLogs, criteria, customerId }) {
  const client = new Anthropic();

  const res = await client.messages.create({
    model: 'claude-sonnet-4-20250514', max_tokens: 4096,
    system: 'Evaluate evidence vs acceptance criteria. Return JSON:\n' +
      '{ decision: "go"|"no-go"|"conditional",\n' +
      '  criteria_results: [{ criterion, met, evidence }],\n' +
      '  outstanding_items: [{ item, blocking, action }] }',
    messages: [{ role: 'user', content:
      `Criteria: ${JSON.stringify(criteria)}\n` +
      `Tests: ${JSON.stringify(testResults)}\n` +
      `Runbook: ${JSON.stringify(runbookLogs)}`
    }]
  });

  const validation = parseJSON(res);
  await generateEvidencePDF({ validation, customerId });
  return validation;
}
```

REUSABILITY DESIGN

■ Acceptance criteria are per-product with customer-specific overrides. The validator and evidence package generator are fully shared. Every customer receives a consistent, branded evidence package.

DEPENDENCIES

@anthropic-ai/sdk	acceptance_criteria.js on	Smoke Test + Runbook outputs	PDF generator
-------------------	------------------------------	---------------------------------	---------------

The first 30 days post-go-live shape the customer relationship. AI-driven operations means you detect and resolve issues before the customer even notices.

BL

Baseline Learner Agent

Learns what "normal" looks like for each customer deployment in week 1, establishing statistical ...

INPUTS	OUTPUTS
■ 7-day metrics window (CPU, memory, latency, error rates, throughput)	■ Baseline profile JSON with statistical bounds per metric
■ Deployment metadata (product version, env type, user count)	■ Anomaly detection thresholds (warn at 2σ , alert at 3σ)
	■ Metric correlation map

CODE SNIPPET · Claude Code (claude-sonnet-4-20250514)

```
// baseline_learner.js
export async function learnBaseline({ customerId, deploymentMeta }) {
  const client = new Anthropic();
  const metrics = await queryMetrics({ customerId, windowDays: 7 });

  const res = await client.messages.create({
    model: 'claude-sonnet-4-20250514', max_tokens: 2048,
    system: 'Analyse this metrics window and establish baseline profiles.\n' +
      'For each metric: mean, p50, p95, p99, std_dev, thresholds.\n' +
      'Identify correlations. Return JSON baseline profile.',
    messages: [{ role: 'user', content:
      `Metrics: ${JSON.stringify(metrics)}\n` +
      `Deployment meta: ${JSON.stringify(deploymentMeta)}`
    }]
  });

  const baseline = parseJSON(res);
  await storeBaseline(customerId, baseline);
  await scheduleWeeklyRefresh(customerId);
  return baseline;
}
```

REUSABILITY DESIGN

- Metric schema is standardised — the same schema works for all customers and products. Thresholds are per-customer learned values, not hard-coded. The baseline is refreshed weekly.

DEPENDENCIES

@anthropic-ai/sdk	Metrics platform (Datadog / Prometheus)	Time-series DB	Baseline store
-------------------	--	----------------	----------------

AD

Anomaly Detector Agent

Continuously compares real-time metrics against the learned baseline and raises proactive alerts ...

INPUTS	OUTPUTS
■ Real-time metrics stream	■ Alert events (severity, affected component, context, recommended action)
■ Customer baseline profile	■ Anomaly timeline
■ Alert routing config	■ Correlation with recent changes

CODE SNIPPET · Claude Code (claude-sonnet-4-20250514)

```
// anomaly_detector.js
export async function detectAnomalies({ currentMetrics, baselineProfile, customerId }) {
  const client = new Anthropic();

  // Fast path: statistical threshold check
  const candidates = checkThresholds(currentMetrics, baselineProfile);
  if (candidates.length === 0) return [];

  // AI path: enrich confirmed anomalies with context
  const recentChanges = await getRecentChanges(customerId);

  const res = await client.messages.create({
    model: 'claude-sonnet-4-20250514', max_tokens: 1024,
    system: 'For each candidate anomaly: confirm if genuine, assess' +
      ' severity (info/warn/critical), suggest probable cause.' +
      ' Return JSON array.',
    messages: [{ role: 'user', content:
      `Candidates: ${JSON.stringify(candidates)}\n` +
      `Baseline: ${JSON.stringify(baselineProfile)}\n` +
      `Recent changes: ${JSON.stringify(recentChanges)}`
    }]
  });

  const confirmed = parseJSON(res);
  await routeAlerts(confirmed, customerId);
  return confirmed;
}
```

REUSABILITY DESIGN

- Detection logic and alert routing are fully reusable. Baseline profiles are per-customer (auto-learned). Alert routing configs are per-customer. The detector runs on a shared polling scheduler.

DEPENDENCIES

@anthropic-ai/sdk	Baseline Learner output	Metrics stream	Notification service (Slack / PagerDuty)
-------------------	----------------------------	----------------	---

SH

Self-Healer Agent

Auto-remediates known failure patterns within a configurable policy scope. Gets smarter as the re...

INPUTS	OUTPUTS
■ Anomaly alert events	■ Remediation actions taken (structured log)
■ Remediation playbook library	■ Result status (fixed / escalated / pending approval)
■ Approval policy (auto-fix vs. human sign-off)	■ Escalation pack if remediation failed

CODE SNIPPET · Claude Code (claude-sonnet-4-20250514)

```
// self_healer.js
export async function attemptRemediation({ anomaly, customerId, policy }) {
  const client = new Anthropic();
  const playbooks = await loadPlaybooks(anomaly.product_id);

  // Step 1: Select best playbook
  const sel = await client.messages.create({
    model: 'claude-sonnet-4-20250514', max_tokens: 512,
    system: 'Select best remediation playbook. Return JSON:' +
      ' { playbook_id, confidence, rationale }',
    messages: [{ role: 'user', content:
      `Anomaly: ${JSON.stringify(anomaly)}`
    }]
  });

  const { playbook_id, confidence } = parseJSON(sel);
  if (!playbook_id || confidence < 0.8) return escalate(anomaly, customerId);

  // Step 2: Check approval policy
  const pb = playbooks.find(p => p.id === playbook_id);
  if (!policy.auto_approve.includes(pb.category))
    return requestRemediationApproval(pb, anomaly, customerId);

  // Step 3: Execute and verify
  await executePlaybook(pb, anomaly.context);
  return verifyRemediation(anomaly, customerId);
}
```

REUSABILITY DESIGN

- Playbooks are per-product and grow with every incident resolved. Approval policies are per-customer. The execution engine is fully shared.

DEPENDENCIES

AI-first support that is contextually richer than traditional L1 — because it knows the exact customer environment. Every resolved ticket becomes a knowledge asset.

HD

L1/L2 Helpdesk Agent

Handles customer queries with full deployment context injected. Resolves or escalates with a comp...

INPUTS	OUTPUTS
■ Customer query (text / ticket)	■ Resolution response or escalation
■ Customer deployment config	■ Confidence score
■ Relevant KB articles	■ KB articles used
■ Recent anomaly and ticket history	■ Full escalation context pack (if escalated)

CODE SNIPPET · Claude Code (claude-sonnet-4-20250514)

```
// helpdesk_agent.js
export async function handleQuery({ query, customerId }) {
  const client = new Anthropic();

  const [config, kb, anomalies, tickets] = await Promise.all([
    loadDeploymentConfig(customerId),
    searchKnowledgeBase(query, customerId),
    getRecentAnomalies(customerId, { days: 7 }),
    getRecentTickets(customerId, { days: 30 })
  ]);

  const res = await client.messages.create({
    model: 'claude-sonnet-4-20250514', max_tokens: 2048,
    system: 'You are a deployment support specialist.' +
      ' Return JSON: { resolved, response, confidence, kb_used[] }',
    messages: [{ role: 'user', content:
      `Query: ${query}\nConfig: ${JSON.stringify(config)}\n` +
      `KB: ${JSON.stringify(kb)}\nAnomalies: ${JSON.stringify(anomalies)}`
    }]
  });

  const result = parseJSON(res);
  if (result.resolved && result.confidence > 0.85)
    await draftKBArticle(query, result.response, config.product_id);
  return result;
}
```

REUSABILITY DESIGN

- Core agent is fully shared. Context is injected per-customer. KB articles are per-product × version and grow with every resolved ticket.

DEPENDENCIES

@anthropic-ai/sdk	Customer deployment config	Knowledge base	Anomaly history	Ticket history
-------------------	----------------------------	----------------	-----------------	----------------

TC

Ticket Classifier Agent

Classifies incoming tickets by category and severity, enriches with deployment context and routes...

INPUTS	OUTPUTS
■ Incoming ticket text	■ Category (config / performance / integration / data / security)
■ Customer ID	■ Severity (P1–P4)
■ Routing rules config	■ Assigned team / SME
	■ Context enrichment

CODE SNIPPET · Claude Code (claude-sonnet-4-20250514)

```
// ticket_classifier.js
export async function classifyAndRoute({ ticket, customerId }) {
  const client = new Anthropic();
  const meta = await getDeploymentMetadata(customerId);

  const res = await client.messages.create({
    model: 'claude-sonnet-4-20250514', max_tokens: 512,
    system: 'Classify this ticket. Return ONLY JSON:\n' +
      '{ category: "config|performance|integration|data|security",\n' +
      ' severity: "P1|P2|P3|P4",\n' +
      ' component, confidence, enrichment }',
    messages: [{ role: 'user', content:
      `Ticket: ${ticket.text}\n` +
      `Product: ${meta.product} v${meta.version}`
    }]
  });

  const classification = parseJSON(res);
  const routing = applyRoutingRules(classification, await loadRoutingRules());
  return { ...classification, ...routing };
}
```

REUSABILITY DESIGN

■ Classification taxonomy is per-product. Routing rules are configurable per organisation. The classifier engine is shared.

DEPENDENCIES

@anthropic-ai/sdk	Routing rules config	Deployment metadata	Historical tickets DB
-------------------	----------------------	---------------------	-----------------------

RC

RCA Engine Agent

Performs root cause analysis for production incidents by correlating logs, metrics, change histor...

INPUTS	OUTPUTS
■ Incident details	■ RCA report (probable cause, contributing factors, confidence)
■ Application logs (last 6 hours)	■ Timeline of events
■ Metric anomaly timeline	■ Recommended fix + prevention steps
■ Recent deployment / config changes (last 3 days)	■ KB article draft

CODE SNIPPET · Claude Code (claude-sonnet-4-20250514)

```
// rca_engine.js
export async function performRCA({ incidentId, customerId }) {
  const client = new Anthropic();
  const incident = await getIncident(incidentId);

  const [logs, metrics, changes] = await Promise.all([
    getLogs(customerId, { hours: 6, around: incident.started_at }),
    getAnomalyTimeline(customerId, { hours: 6, around: incident.started_at }),
    getChangeHistory(customerId, { days: 3, before: incident.started_at })
  ]);

  const res = await client.messages.create({
    model: 'claude-sonnet-4-20250514', max_tokens: 4096,
    system: 'Perform root cause analysis. Return JSON:\n' +
      '{ probable_cause, confidence, contributing_factors[],\n' +
      '  timeline[{ time, event }], recommended_fix,\n' +
      '  kb_article_draft: { title, summary, steps[] } }',
    messages: [{ role: 'user', content:
      `Incident: ${JSON.stringify(incident)}\n` +
      `Logs (4k): ${logs.slice(0, 4000)}\n` +
      `Anomalies: ${JSON.stringify(metrics)}`
    }]
  });

  const rca = parseJSON(res);
  await draftKBArticle(rca.kb_article_draft, incidentId);
  return rca;
}
```

REUSABILITY DESIGN

- Fully reusable across products and customers. Evidence gathering uses standardised schemas. The KB article draft from every RCA feeds the Helpdesk Agent — the system compounds with every incident.

DEPENDENCIES

@anthropic-ai/sdk	Log aggregation system	Metrics store	Change history log	Knowledge base
-------------------	------------------------	---------------	--------------------	----------------

Getting it up and running

A 16-week plan to build and deploy the full AI Deployment as a Service platform. Each phase delivers working, testable agents. By week 16 you should have your first production deployment complete and a TTV baseline to improve from.

PHASE 00

Foundation

Weeks 1–2

- Setup Claude Code + Anthropic SDK environment
- Define ALL standard schemas up-front (env fingerprint, deployment plan, metric, ticket)
- Build accelerator library folder structure + versioning conventions
- Setup CI/CD pipeline for agent testing with mock data
- Define and integrate approval workflow (Slack / email)
- Establish historical deployment data store schema

PHASE 01

Pre-Deploy + Deploy

Weeks 3–6

- Build Environment Scanner Agent (AWS + Azure adapters first)
- Build Dependency Mapper Agent
- Build Infra Sizer Agent + first product benchmark configs
- Build Config Adapter Agent + first product config templates
- Build IaC Generator Agent (Terraform, AWS first)
- Build Provisioning Orchestrator with retry / rollback logic
- Create first 3 product IaC module catalogs
- End-to-end test: pre-deploy → deploy on test environment

PHASE 02

Validate + Operate

Weeks 7–10

- Build Smoke Test Runner Agent
- Build Runbook Executor Agent + create first runbook library
- Build Sign-off Gate Validator + evidence package PDF generator
- Build Baseline Learner Agent
- Build Anomaly Detector Agent + configure alert routing
- Build Self-Healer Agent + define first 5 auto-remediation playbooks
- Wire all 9 agents into the full deployment orchestration pipeline

PHASE 03

Support + Go-Live

Weeks 11–16

- Build L1/L2 Helpdesk Agent + seed KB with known issues
- Build Ticket Classifier + routing rules engine
- Build RCA Engine Agent
- Integrate feedback loops (resolved tickets → KB → Helpdesk)
- Build TTV analytics dashboard (by phase, product, customer)
- Run full pipeline against test customer environment (shadow mode)
- Measure TTV baseline and fine-tune accelerator library
- Go live with first production deployment — measure and iterate

KEY IMPLEMENTATION PRINCIPLES

Schema-first

Define all input/output schemas before writing any agent code. Fixed schemas are what make agents composable, testable and reusable across products.

Agent isolation

Each agent works standalone with mock inputs. Never couple agents directly — always communicate via structured JSON files or APIs. This enables independent testing.

Config-driven products Product-specific knowledge (requirements, benchmarks, templates) lives in config files, not agent code. Adding a new product = new config directory, zero code change.	Feedback loops from day 1 Build the pipeline that feeds deployment outcomes back into the accelerator library from the very first deployment. This compound effect makes the service defensible.
Human gates are features Approval gates are not technical debt — they are trust-building checkpoints with customers. Design them to be lightweight, mobile-friendly and logged.	Measure TTV continuously Instrument every phase from day 1. TTV per phase is the primary KPI — it should be falling with every successive deployment on the same product.